

AI CODING ASSISGNMENT 7:4

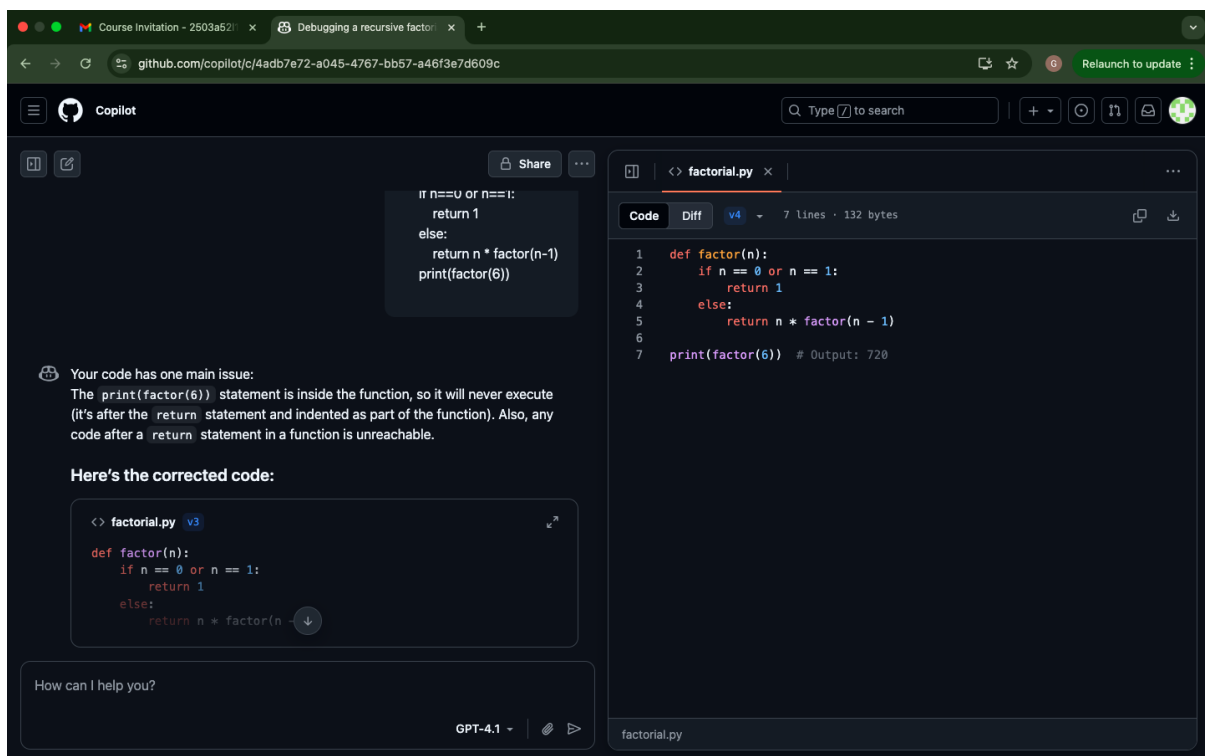
Htno: 2503A52L15

VARSHITH REDDY

Task Description #1:

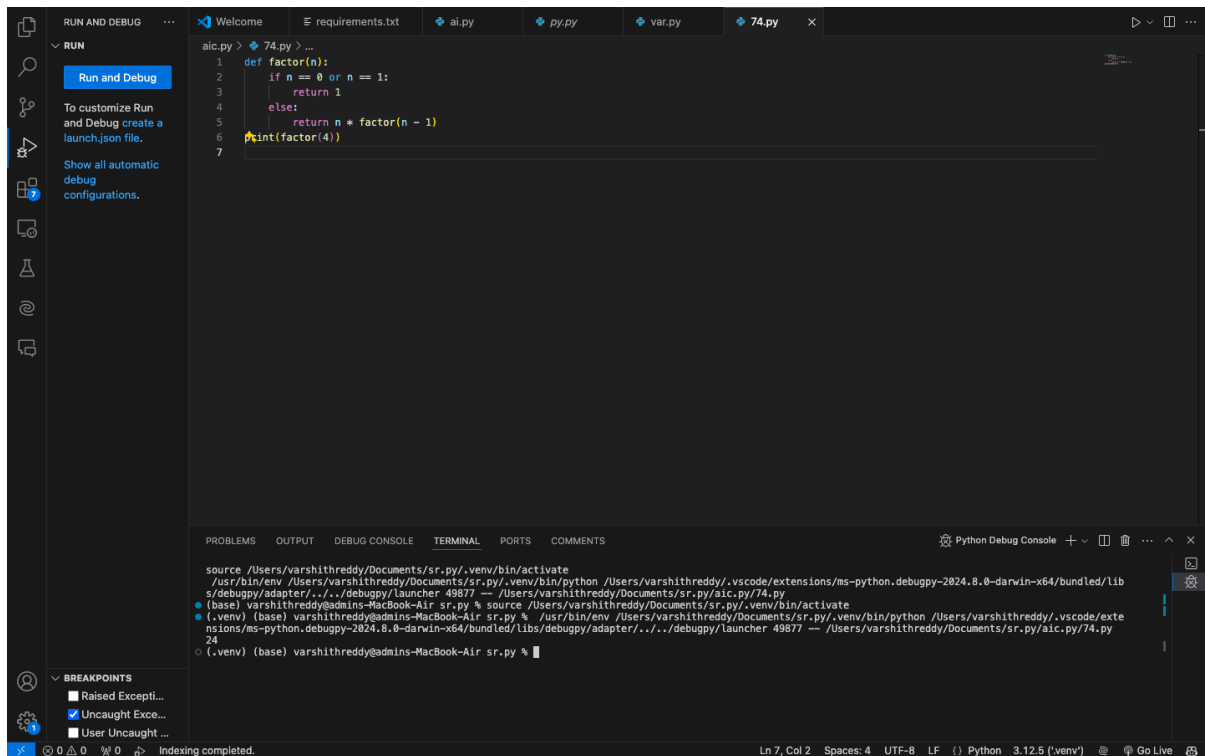
- Introduce a buggy Python function that calculates the factorial of a number using recursion.

Use Copilot or Cursor AI to detect and fix the logical or syntax errors.



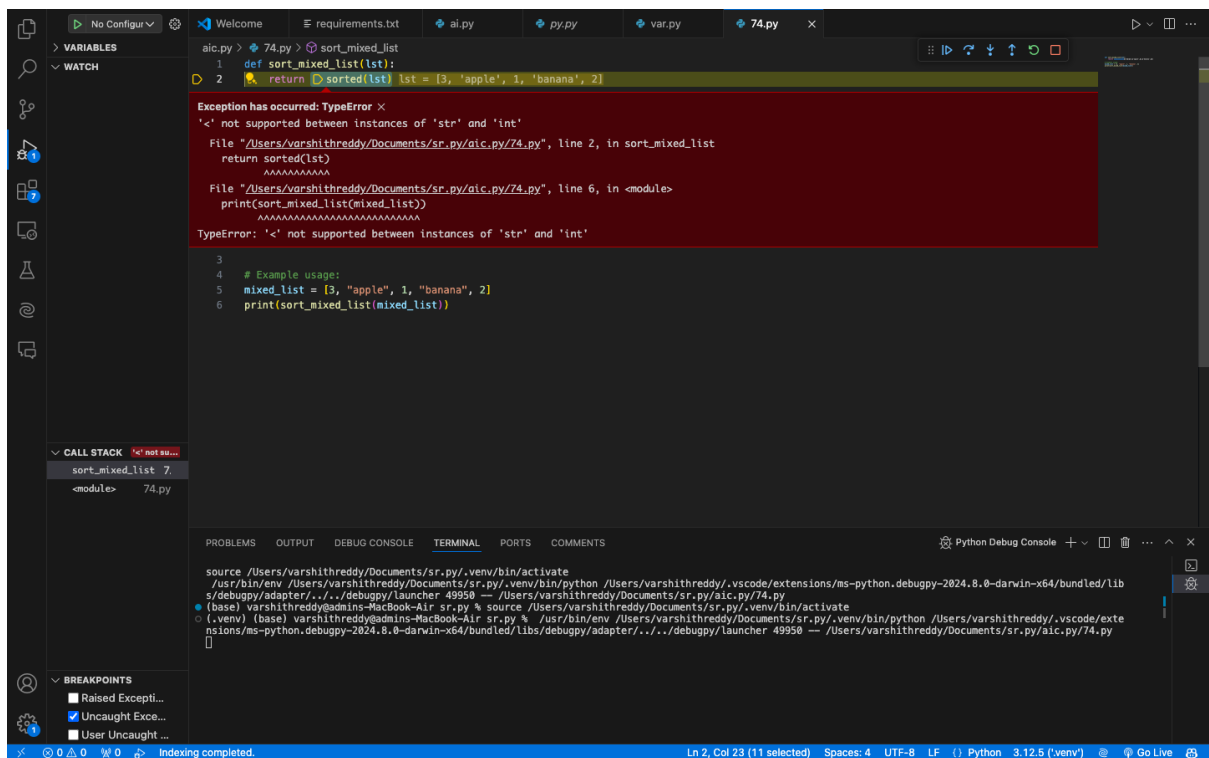
Expected Outcome #1:

- Copilot or Cursor AI correctly identifies missing base condition or incorrect recursive call and suggests a functional factorial implementation



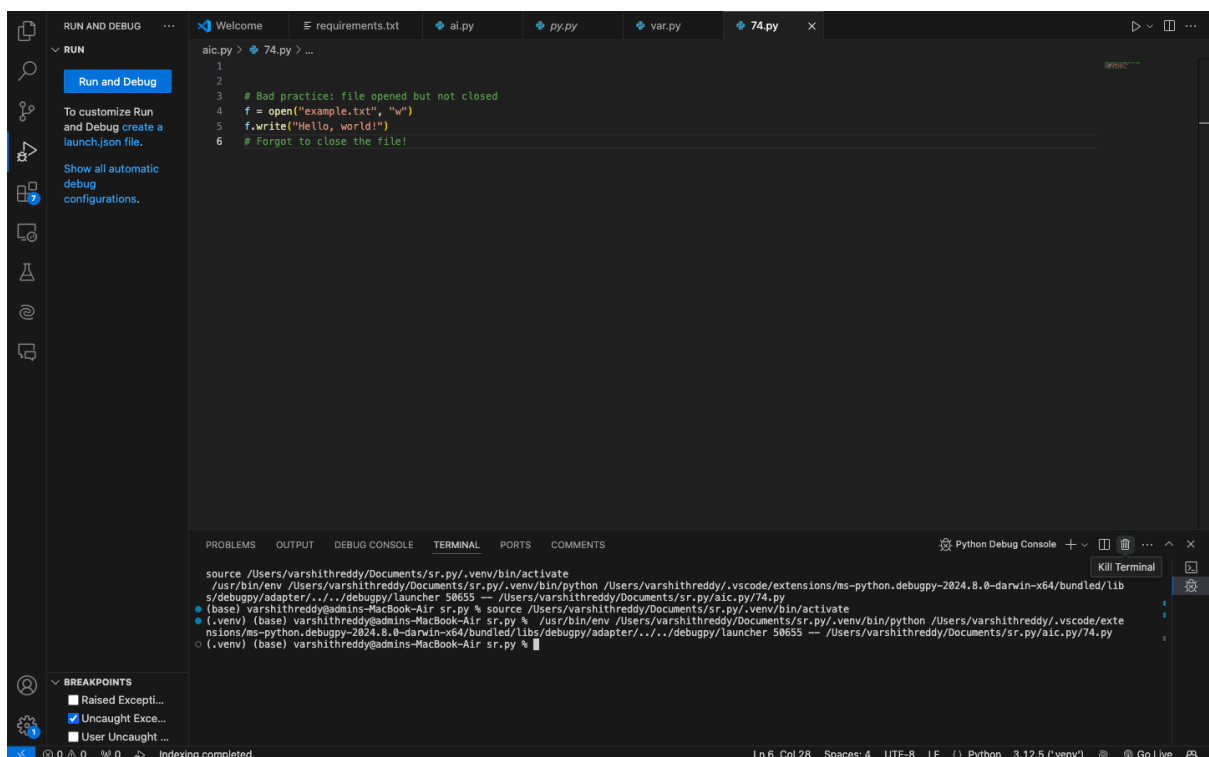
Task Description #2:

- Provide a list sorting function that fails due to a type error (e.g., sorting list with mixed integers and strings). Prompt AI to detect the issue and fix the code for consistent sorting.



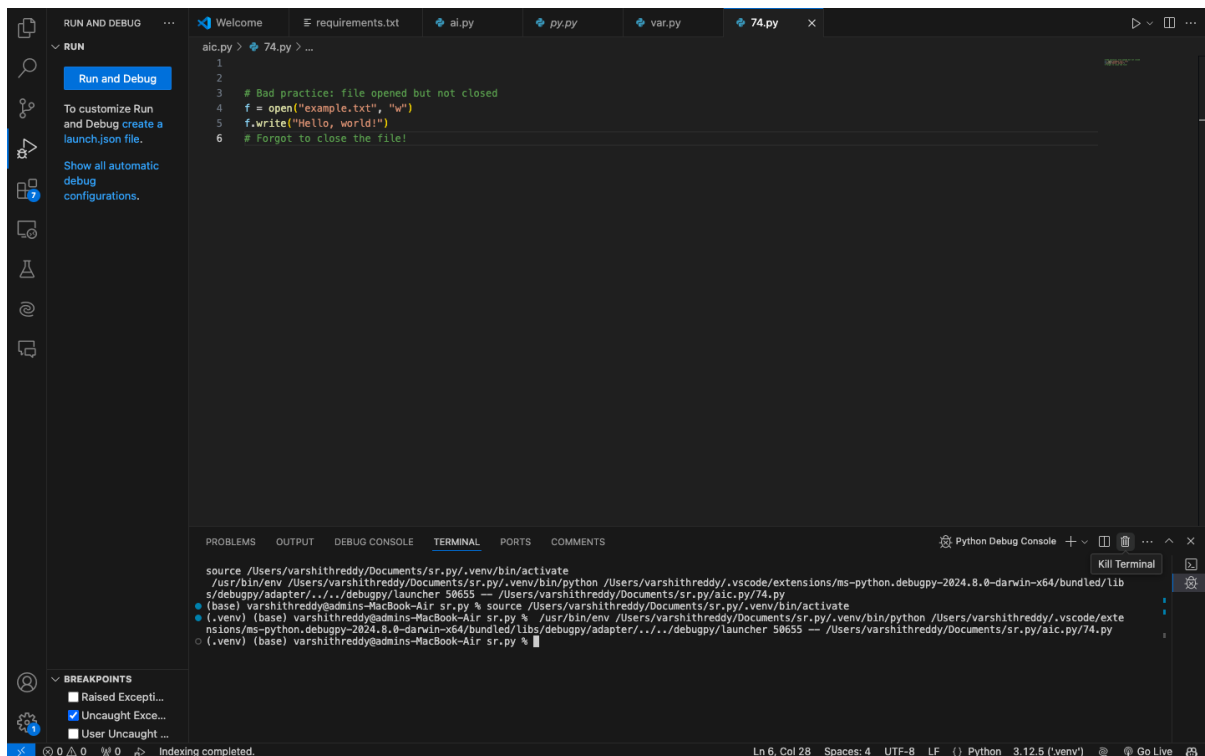
Output:

AI detects the type inconsistency and either filters or converts list elements, ensuring successful sorting without a crash



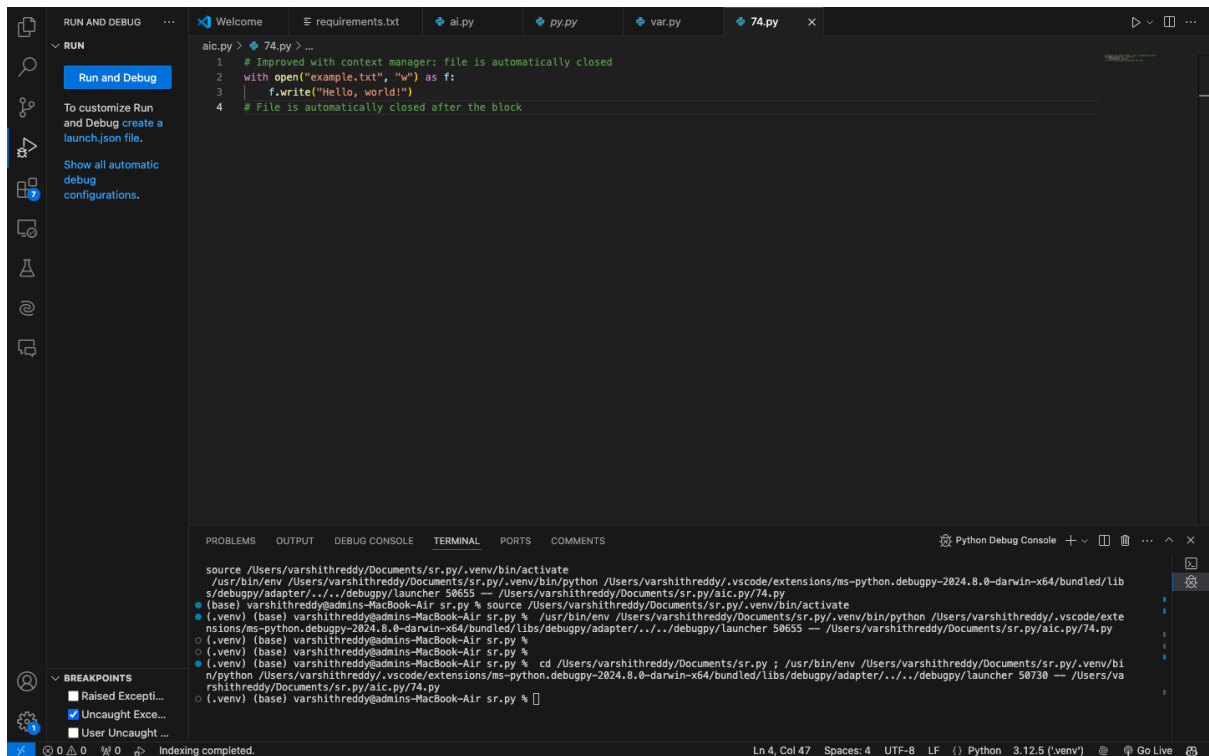
Task Description #3:

- Write a Python snippet for file handling that opens a file but forgets to close it. Ask Copilot or Cursor AI to improve it using the best practice (e.g., with open() block).



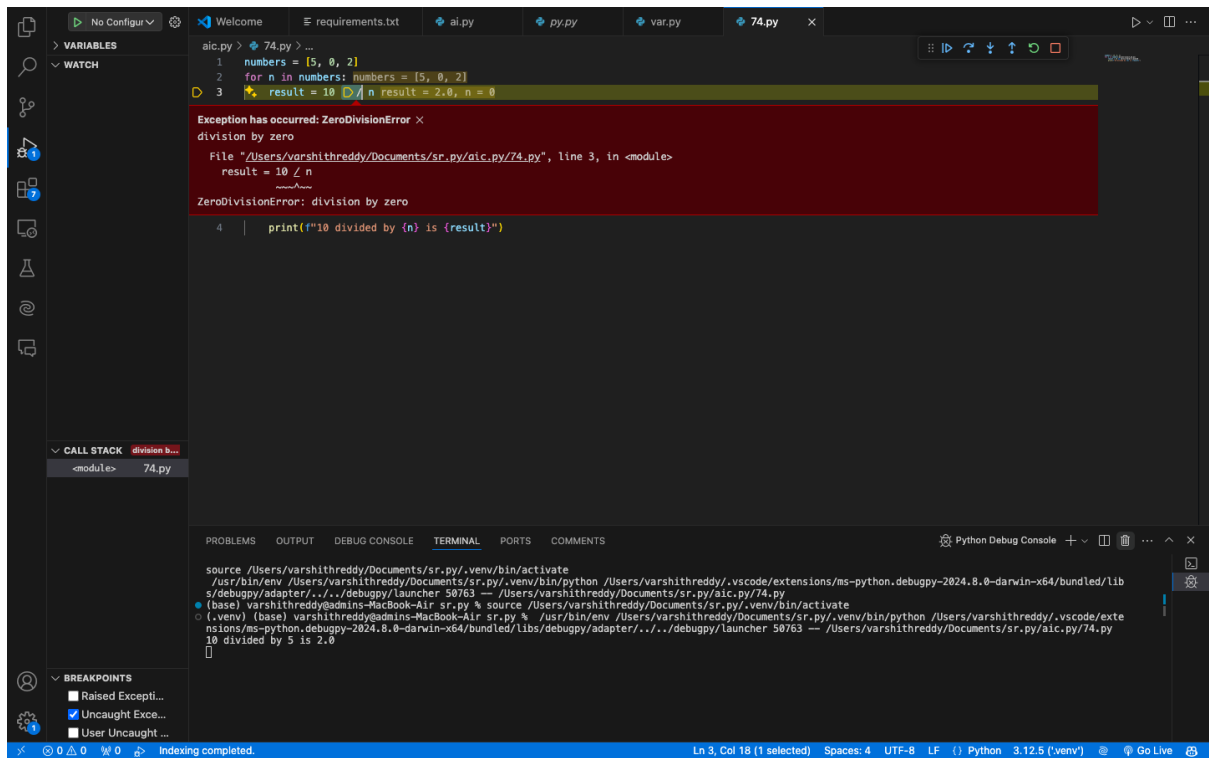
Expected Outcome #3:

- AI refactors the code to use a context manager, preventing resource leakage and runtime warnings



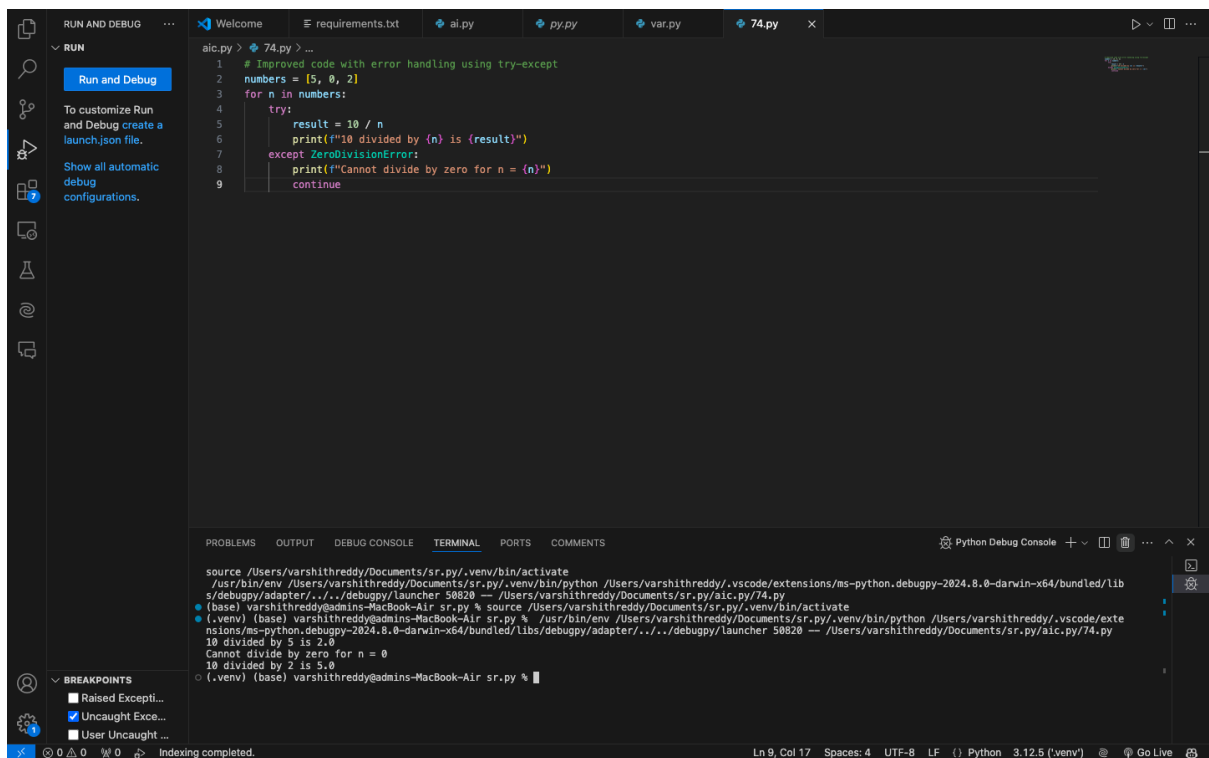
Task Description #4:

- Provide a piece of code with a ZeroDivisionError inside a loop. Ask AI to add error handling using try-except and continue execution safely



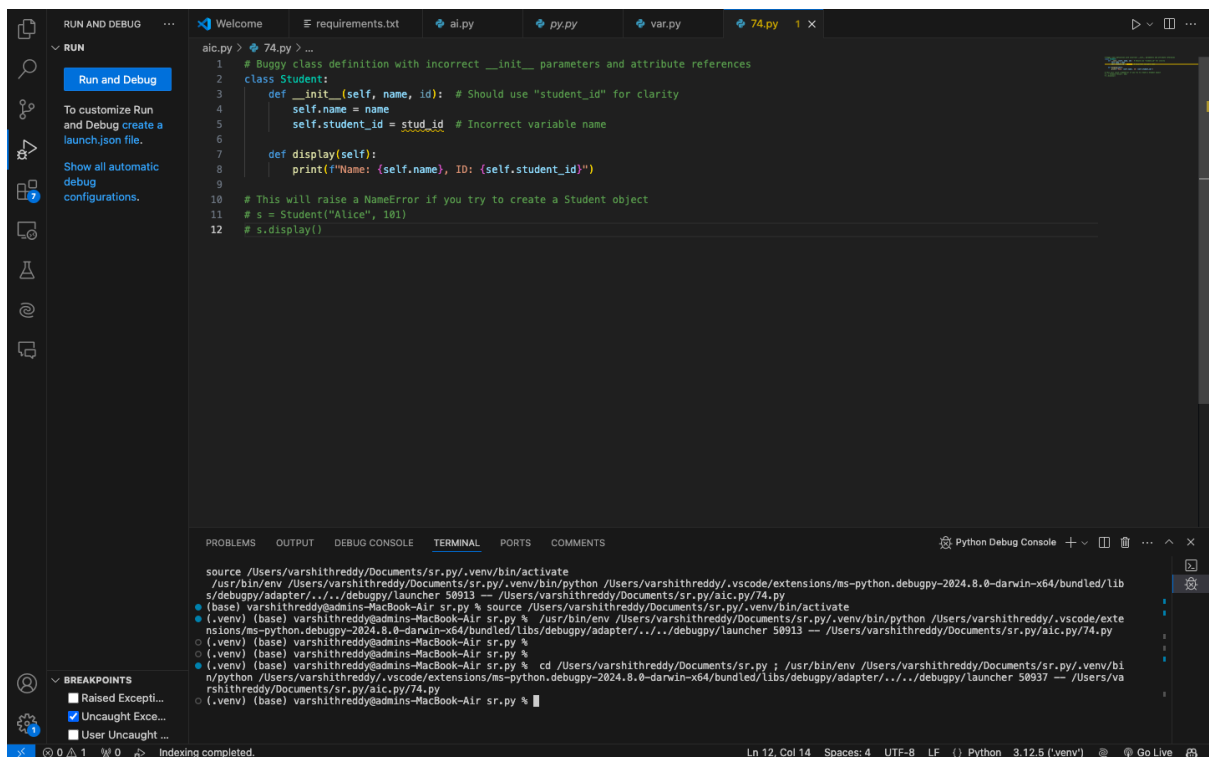
Expected Outcome #4:

- Copilot adds a try-except block around the risky operation, preventing crashes and printing a meaningful error message.



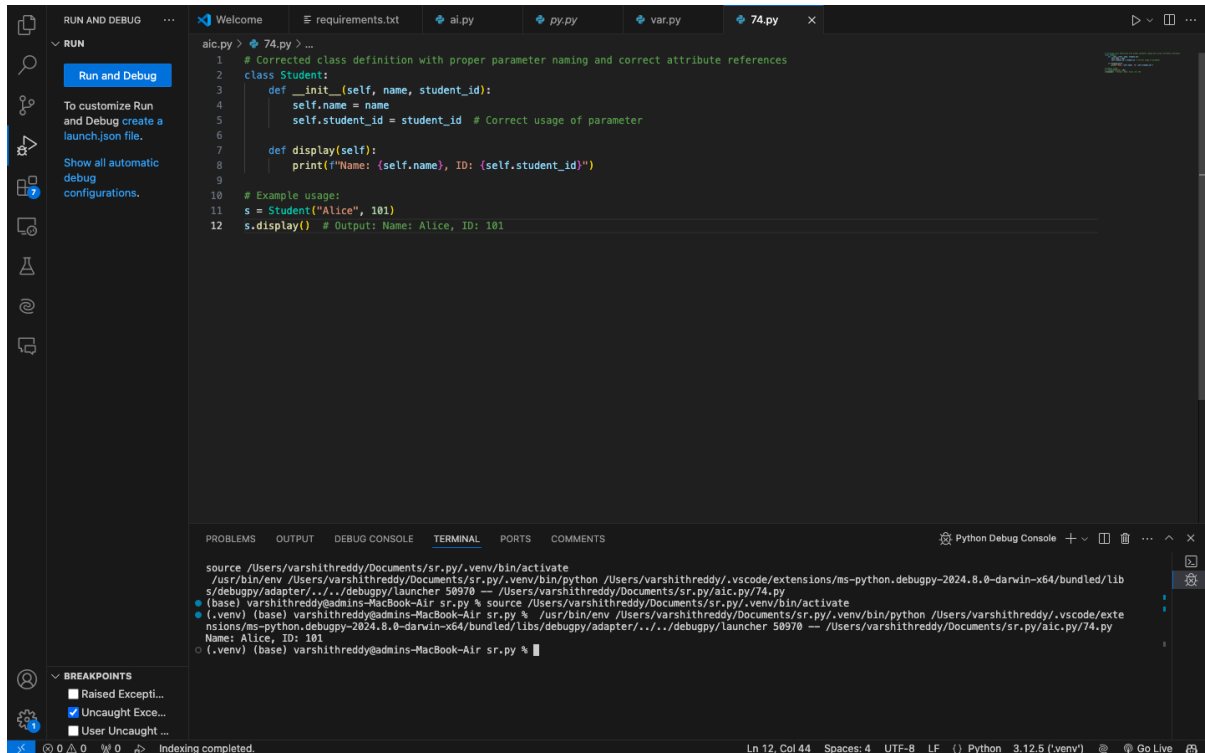
Task Description #5:

- Include a buggy class definition with incorrect `__init__` parameters or attribute references. Ask AI to analyze and correct the constructor and attribute usage



Expected Outcome #5:

- Copilot identifies mismatched parameters or missing self references and rewrites the class with accurate initialization and usage



The screenshot displays the Visual Studio Code interface with a Python file named `74.py` open. The code defines a `Student` class with an `__init__` method and a `display` method. The `__init__` method takes `name` and `student_id` as parameters and assigns them to `self.name` and `self.student_id` respectively. The `display` method prints the name and ID. An example usage is provided, creating a `Student` object named `s` with the name "Alice" and ID 101, and then calling `s.display()`.

```
1 # Corrected class definition with proper parameter naming and correct attribute references
2 class Student:
3     def __init__(self, name, student_id):
4         self.name = name
5         self.student_id = student_id # Correct usage of parameter
6
7     def display(self):
8         print(f"Name: {self.name}, ID: {self.student_id}")
9
10 # Example usage:
11 s = Student("Alice", 101)
12 s.display() # Output: Name: Alice, ID: 101
```

The terminal output shows the execution of the script, confirming the output: `Name: Alice, ID: 101`.

```
source /Users/varshithreddy/Documents/sr.py/.venv/bin/activate
/usr/bin/env /Users/varshithreddy/Documents/sr.py/.venv/bin/python /Users/varshithreddy/.vscode/extensions/ms-python.debugpy-2024.8.0-darwin-x64/bundled/lib
s/debugpy/adapter/../../debugpy/launcher 58970 -- /Users/varshithreddy/Documents/sr.py/74.py
(.venv) varshithreddy@admins-MacBook-Air sr.py % source /Users/varshithreddy/Documents/sr.py/.venv/bin/activate
(.venv) (base) varshithreddy@admins-MacBook-Air sr.py % /usr/bin/env /Users/varshithreddy/Documents/sr.py/.venv/bin/python /Users/varshithreddy/.vscode/exte
nsions/ms-python.debugpy-2024.8.0-darwin-x64/bundled/lib/debugpy/launcher 58970 -- /Users/varshithreddy/Documents/sr.py/aic.py/74.py
Name: Alice, ID: 101
(.venv) (base) varshithreddy@admins-MacBook-Air sr.py %
```